以下是**四个完整的 Markdown 文档**。请将每个文档保存到指定路径下`/mnt/user-data/outputs/`。这些文档相互引用，旨在由 AI 代理直接执行。

==========

文件 1 —`/mnt/user-data/outputs/Phase1a_R2_Validation_Protocol.md`

==========

第一阶段 R2 验证方案

所有者：Scarlettfw / iDP3-UAV **状态**：可由自主代理执行 **触发条件**：第一阶段 R2 训练运行完成（损失平台期 ≥ 30 个 epoch 或步数 ≥ 计划最大值） **预计耗时**：1 个工作日（约 8 小时）

1.文档目的与决策标准

该协议决定**阶段 1a R2**（KD 树障碍排斥 + 末端权重重采样，R2 修复 3/4 + 优化 1/2）是否可以**作为阶段 1b 热启动检查点发布**，或者需要对 R2 特定旋钮（`end_weight`, , `obstacle_safe_margin`归一化器边界）进行进一步迭代。

阶段 1a R2 收官硬性标准（所有门必须同时保持）：

门	指标	临界点	源测试
G1	线下推广活动成功	≥ 95% (≥ 19/20)	测试 1
G2	N=2绕行比率	≤ 1.40（与旧的 1.27 基线相比；≤ 10% 的退化）	测试 2
G3	N=4 平均绕行比率	< 1.50（旧值：2.65–3.20）	测试 3
G4	N=4 首次之后的平均最大超调 `dist<1m`	≤ 1.0 米（旧：4.61 米）	测试 3
G5	N=4 末段（`dist<1m`）raw_dx 输出平均值	< 0.15 米/步（旧值：0.293 米，饱和值 ≈ 0.1875 归一化器上限）	测试 3
G6	多障碍 clr<0.5 份额	< 1.5%（旧：3.3%）且 0 帧 clr<0.2	测试 4
G7	20个世界闭环成功案例	≥ 70%（旧：约 50%）	测试 5

中止标准（停止协议并升级）：

- 测试 1 成功率 < 80% → 检查点损坏或规范化器不匹配；请重新运行`--check-norm`。
- 测试 2 绕行 > 2.0 在简单的 N=2 几何中 → 直线路径规划出现灾难性倒退；恢复`end_weight`到 10 倍。

2.前置检查清单（可执行bash）

```
狂欢
```

```
# 2.1 Checkpoint 存在性
CKPT_DIR=~/AUV/Improved-3D-Diffusion-Policy-main/Improved-3D-Diffusion-Policy/data/o
ls -lh $CKPT_DIR/checkpoints/latest.ckpt $CKPT_DIR/.hydra/config.yaml
test -s $CKPT_DIR/checkpoints/latest.ckpt || { echo "[FAIL] ckpt missing"; exit 1; }

# 2.2 训练 config 关键参数确认 (R2 fixes 应已 baked-in)
grep -E "end_weight|obstacle_safe_margin|n_obs_steps|n_action_steps|horizon|cfg_drop
    $CKPT_DIR/.hydra/config.yaml
# 期望:
#   n_obs_steps: 3, n_action_steps: 8, horizon: 16
#   cfg_dropout: 0.15, cfg_lambda: 1.5
#   end_weight: <10|20|30>          # R2 候选, 须记录实际值
#   obstacle_safe_margin: 0.5

# 2.3 部署侧 ml_planner_gazebo.py 同步性检查
PLANNER=~/AUV/catkin_ws0/src/ego-planner-swarm/src/planner/plan_manage/scripts/ml_pl
grep -nE "obstacle_safe_margin|_project_trajectory|_obstacle_project|z_max|Rz.*yaw"
grep -nE "deployment_action_steps" $PLANNER   # 应为 2 / 4 / 8 之一

# 2.4 launch 文件
LAUNCH=~/AUV/catkin_ws0/src/ego-planner-swarm/src/planner/plan_manage/launch/single_
grep -E "obs_num|target_x|target_y|target_z|z_max|goal_tolerance" $LAUNCH

# 2.5 Gazebo / PX4 sanity
[ -z "$(pgrep -f roscore)" ] || { echo "[WARN] roscore already running, kill it"; }
which gazebo && gazebo --version | head -1
test -d ~/PX4-Autopilot || { echo "[FAIL] PX4-Autopilot not found"; exit 1; }

# 2.6 输出目录
RUN_TAG=$(date +%Y%m%d_%H%M%S)_p1a_r2
mkdir -p ~/ml_planner_log/$RUN_TAG/{rollout_offline,n2,n4,multi,closed}
echo "$RUN_TAG" > ~/ml_planner_log/RUN_TAG
echo "RUN_TAG=$RUN_TAG" | tee ~/ml_planner_log/$RUN_TAG/env.txt
```

如果`[FAIL]`发生上述任何火灾，**请勿继续操作**。

---

3. 基线对比快照（旧Phase 1a）

所有数值均由用户独立验证。

指标	旧阶段 1a 值	R2靶标	方向
线下推广活动成功	100% (20/20)	≥ 95%	保存
N=2 路径长度（运行 205730）	9.70米	≤ 10.7 米	保存
N=2 位移	7.61米	≈ 7.6 米	保存
N=2绕行比率	1.27	≤ 1.40	保存
N=2 持续时间	125秒	≤ 140 秒	保存
N=4 路径（运行 120958，第 1 次）	24.83米	小于 12 米	**必须改进**
N=4 绕行（运行 120958）	3.20	< 1.5	**必须改进**
N=4 路径（运行 150423，第 3 次）	21.46 米	小于 12 米	**必须改进**
N=4 绕行（运行 150423）	2.65	< 1.5	**必须改进**
N=4 超调后的最终d	0.48–0.49 米	在容差范围内达到目标，无超调	**必须改进**
N=4 最大超调	4.61 米（从 d=0.78 → d=4.61）	≤ 1.0 米	**必须改进**
N=4 末段 raw_dx	0.293 米（饱和归一化器 ≈ 0.1875 上限）	小于 0.15 米	**必须改进**
20个世界闭环	约50%	≥ 70%	提升
clr<0.5 份额	3.3%	< 1.5%	提升
clr<0.2帧	>0	0	提升

末段顶死特征（当 d=0.79 m 时，raw_dx_w = 0.293 m，达到归一化器上限）是“模型无法减速”的最主要诊断故障模式。测试 3 专门针对此模式进行探测。

---

4. 测试 1 — 离线发布 (`eval_phase1a_offline.py`)

4.1 命令

狂欢

```
cd ~/AUV/Improved-3D-Diffusion-Policy-main/Improved-3D-Diffusion-Policy
source ~/miniconda3/etc/profile.d/conda.sh && conda activate idp3
RUN_TAG=$(cat ~/ml_planner_log/RUN_TAG)
python scripts/eval_phase1a_offline.py \
    --ckpt $CKPT_DIR/checkpoints/latest.ckpt \
    --dataset /home/cjj/AUV/avoidance_dataset/merged_body_12.zarr \
    --n_episodes 20 \
    --device cuda:0 \
    --out ~/ml_planner_log/$RUN_TAG/rollout_offline/result.json \
    2>&1 | tee ~/ml_planner_log/$RUN_TAG/rollout_offline/log.txt
```

脚本必须报告`{episodes, success, mean_path_err, mean_terminal_err}`。如果`eval_phase1a_offline.py`不存在，则规范实现会遍历训练时选择的 20 个保留回合，`policy.predict_action(obs_dict)`对每个回合调用，当预测的轨迹终点与记录的终点位置相差在 0.5 米以内，且每步世界帧偏差不超过 1.0 米时，则判定为成功。

4.2 通过标准

结果	决定
⩾ 19/20 (⩾ 95 %)	**通过**——进入测试 2
16–18/20 (80–94%)	**警告**——在继续操作之前，请运行 §10.3（z 轴投影验证）和 §10.4（障碍物投影验证）。
< 16/20 (< 80 %)	**停止**——检查点或归一化器故障。运行并`python scripts/check_normalizer.py --ckpt ...`验证范围是否符合±1.5米的阶跃变化。请勿运行Gazebo实时测试。`data/state` `data/action`

---

5. 测试 2 — 单 AUV N=2 基线保持

N=2 的情况（2 个圆柱体）已由旧的 Phase 1a 干净利落地解决。R2 不应该对其进行回归。

5.1 启动参数 sed

狂欢

```
LAUNCH=~/AUV/catkin_ws0/src/ego-planner-swarm/src/planner/plan_manage/launch/single_
cp $LAUNCH ${LAUNCH}.bak
sed -i 's|<arg name="obs_num" default="[0-9]*"/>|<arg name="obs_num" default="2"/>|'
sed -i 's|<arg name="target_x" default="[0-9.\-]*"/>|<arg name="target_x" default="7
sed -i 's|<arg name="target_y" default="[0-9.\-]*"/>|<arg name="target_y" default="0
sed -i 's|<arg name="target_z" default="[0-9.\-]*"/>|<arg name="target_z" default="2
sed -i 's|<arg name="z_max" default="[0-9.]*"/>|<arg name="z_max" default="3.0"/>|'
diff ${LAUNCH}.bak $LAUNCH
```

5.2 跑

狂欢

```
RUN_TAG=$(cat ~/ml_planner_log/RUN_TAG)
LOG_N2=~/ml_planner_log/$RUN_TAG/n2/run_$(date +%H%M%S)
mkdir -p $LOG_N2
roslaunch ego_planner single_AUV.launch \
    log_dir:=$LOG_N2 use_ml_planner:=true \
    ckpt_path:=$CKPT_DIR/checkpoints/latest.ckpt \
    > $LOG_N2/ros.log 2>&1 &
ROSLAUNCH_PID=$!
sleep 180
kill -INT $ROSLAUNCH_PID; wait $ROSLAUNCH_PID 2>/dev/null
rosnode kill -a 2>/dev/null; killall -9 gzserver gzclient px4 2>/dev/null
```

5.3 指标提取

狂欢

```
python ~/ml_planner_log/scripts/analyze_n4_test.py --run_dir $LOG_N2 --mode n2 \
    --baseline_path 9.70 --baseline_disp 7.61 --baseline_detour 1.27 --baseline_dur
```

分析器读取`pos_cmd_<TS>.csv`并`diffusion_<TS>.csv`计算 path_len、位移`detour = path_len/disp`、持续时间、`duty_traj`（pos_cmd 非过时的刻度比例）。

5.4 通过标准

- `success == True` AND `detour ⩽ 1.40` AND `duration ⩽ 140 s` AND `duty_traj > 0.85` → **G2 PASS**。
- `1.40 < detour ⩽ 1.80` → 警告（记录，进行测试 3；最终船舶决定推迟）。
- `detour > 1.80` 或者 `success == False` → **停止**：`end_weight` 正在扭曲直线路径规划。重新训练 R2，并将`end_weight`参数降低至 10 倍左右。

---

6. 测试 3 — single_AUV N=4 ×5 重复（核心测试）

该测试诊断出旧版 Phase 1a 的主要缺陷——末段顶死故障。五次重复测试可区分模型系统误差和 Gazebo 噪声。

6.1 启动 sed + loop

狂欢

```
LAUNCH=~/AUV/catkin_ws0/src/ego-planner-swarm/src/planner/plan_manage/launch/single_
sed -i 's|<arg name="obs_num" default="[0-9]*"/>|<arg name="obs_num" default="4"/>|'
sed -i 's|<arg name="target_x" default="[0-9.\-]*"/>|<arg name="target_x" default="8

RUN_TAG=$(cat ~/ml_planner_log/RUN_TAG)
for i in 1 2 3 4 5; do
    LOG_I=~/ml_planner_log/$RUN_TAG/n4/run${i}_$(date +%H%M%S)
    mkdir -p $LOG_I
    echo "[N=4] repeat $i → $LOG_I"
    roslaunch ego_planner single_AUV.launch \
        log_dir:=$LOG_I use_ml_planner:=true \
        ckpt_path:=$CKPT_DIR/checkpoints/latest.ckpt \
        > $LOG_I/ros.log 2>&1 &
    RPID=$!; sleep 200
    kill -INT $RPID; wait $RPID 2>/dev/null
    sleep 5
    rosnode kill -a 2>/dev/null; killall -9 gzserver gzclient px4 2>/dev/null
    sleep 8
done
```

6.2 汇总分析

狂欢

```
python ~/ml_planner_log/scripts/analyze_n4_test.py \
    --run_dir ~/ml_planner_log/$RUN_TAG/n4 --mode n4_x5 \
    --goal_tol 0.5 \
    --baseline_max_overshoot 4.61 --baseline_detour 3.20 \
    --report ~/ml_planner_log/$RUN_TAG/n4/report.md
```

6.3 `max_overshoot` 定义

每次运行，其中 `dist[t]` = 无人机到目标的欧氏距离（xy 坐标），以每个里程计刻度表示：

1. 查找第一个索引 `t0` 为 `dist[t0] < 1.0 m`.
2. 如果没有这样的 `t0` → `max_overshoot = inf`（运行未到达目标区域）。
3. 否则：`max_overshoot = max(dist[t] for t>t0) − dist[t0]` 如果 t0 之后的最大值超过 `dist[t0]`，否则为 0。
4. 汇报 `mean_max_overshoot` 这5次跑动的情况。

末段的 `raw_dx` 计算方法如下 `diffusion_*.csv`：对于每个对应里程计帧刻度为 的推断 `dist < 1.0 m`，提取 `raw_action[k=0, 0:3]`（第一个 DPOS_W 分量）。报告每次运行的平均值 `|dx|`。

6.4 通过标准

指标	临界点	笔记
成功率	≥ 4/5	深入其中 `goal_tolerance=0.5`，然后停止
平均绕行比率	< 1.5	与旧版 3.20（第一轮）/ 2.65（第三轮）相比
平均最大超调量	≤ 1.0 米	与旧款 4.61 米相比
平均末段 raw_dx	< 0.15 米/步	与旧值 0.293（饱和上限 ≈ 0.1875）相比
平均持续时间	< 100 秒	与旧款 124 s 相比

所有 5 个都必须持有 → **G3、G4、G5 PASS**。

6.5 故障诊断分支表

症状	可能原因	行动
最终段 raw_dx ≥ 0.20 m（仍处于饱和状态）	`end_weight` 太低；末段帧数仍然低于批次的 1%。	使用 `end_weight` ×2 进行重新训练（10→20→30）；参见 §10.5
最后一段 raw_dx 可以，但超出 > 1 m	推理延迟过冲（规划器重新规划速度过慢）	将 8 减 `deployment_action_steps` 4
Detour > 2.0，raw_dx 正常	`obstacle_safe_margin` 太大；路线过于谨慎	降低 `obstacle_safe_margin` 0.5 → 0.3 英寸 `_project_trajectory`
绕行可以，但 clr<0.5 会出现峰值。	部署时未应用障碍物投影	运行 §10.4 — 确认 `rospy.loginfo` 排斥向量
成功率 2-3/5，偶有失败	Gazebo/PX4 初始化竞赛	发射前悬停时间从 3 秒增加到 5 秒
成功率 0/5	归一化器或世界框架错误回归	重新运行测试 1；如果结果仍然是 100%，则说明修复 4（KD 树 obs_dict）已回归——与 R2 提交的差异

7. 测试 4 — 多障碍物间隙分布

7.1 运行 5 个不同的世界

狂欢

```
WORLDS=(world_obs6_a world_obs6_b world_obs8 world_obs10 world_obs12_dense)
RUN_TAG=$(cat ~/ml_planner_log/RUN_TAG)
for W in "${WORLDS[@]}"; do
    LOG_W=~/ml_planner_log/$RUN_TAG/multi/$W
    mkdir -p $LOG_W
    roslaunch ego_planner single_AUV.launch \
        world_name:=$W log_dir:=$LOG_W use_ml_planner:=true \
        ckpt_path:=$CKPT_DIR/checkpoints/latest.ckpt \
        > $LOG_W/ros.log 2>&1 &
    RPID=$!; sleep 220
    kill -INT $RPID; wait $RPID 2>/dev/null
    sleep 5; rosnode kill -a 2>/dev/null
    killall -9 gzserver gzclient px4 2>/dev/null; sleep 8
done
```

7.2 清仓分布

Python

```
# (executed via analyze_n4_test.py --mode clr; inline snippet for clarity)
import pandas as pd, numpy as np, glob
clrs = []
for csv in glob.glob(f"{RUN_DIR}/multi/*/pos_cmd_*.csv"):
    df = pd.read_csv(csv)
    clrs.append(df['min_clr'].dropna().values)
clr_all = np.concatenate(clrs)
share_05 = float((clr_all < 0.5).sum()) / len(clr_all)
share_02 = float((clr_all < 0.2).sum()) / len(clr_all)
print(f"clr<0.5 share = {share_05:.4f}   clr<0.2 frames = {(clr_all<0.2).sum()}")
```

7.3 通过标准

- `share<0.5 < 0.015 (1.5 %)` AND `frames<0.2 == 0` → **G6 PASS**。
- `share<0.5 ∈ [1.5%, 3%]` → 警告，将 `obstacle_safe_margin` 0.5 → 0.6 提高并重新评估（不重新训练）。

- `frames<0.2 > 0`或者`share<0.5 > 3 %`→**停止**：KD 树投影未能影响轨迹。运行 §10.4。

---

8. 测试 5 — 20 世界闭环成功率

8.1 运行

狂欢

```
WORLDS_20=$(ls ~/AUV/catkin_ws0/src/ego-planner-swarm/src/planner/plan_manage/worlds
RUN_TAG=$(cat ~/ml_planner_log/RUN_TAG)
i=0
for W in $WORLDS_20; do
    i=$((i+1)); WN=$(basename $W .world)
    LOG_W=~/ml_planner_log/$RUN_TAG/closed/${i}_${WN}
    mkdir -p $LOG_W
    roslaunch ego_planner single_AUV.launch \
        world_file:=$W log_dir:=$LOG_W use_ml_planner:=true \
        ckpt_path:=$CKPT_DIR/checkpoints/latest.ckpt \
        > $LOG_W/ros.log 2>&1 &
    RPID=$!; sleep 200
    kill -INT $RPID; wait $RPID 2>/dev/null
    rosnode kill -a 2>/dev/null; killall -9 gzserver gzclient px4 2>/dev/null; sleep
done

python ~/ml_planner_log/scripts/analyze_n4_test.py \
    --run_dir ~/ml_planner_log/$RUN_TAG/closed --mode closed20 \
    --report ~/ml_planner_log/$RUN_TAG/closed/report.md
```

8.2 通过标准

- 成功率≥14/20（70%）→ **G7 通过**。
- 10–13/20 (50–65 %) → 警告；在下次决策之前，按世界类型对失败进行聚类分析。
- < 10/20 (< 50 %) →**停止**：R2 的变化未能推广到闭环系统。

---

9. 决策矩阵

`P`= 通过，`W`= 警告，`F`= 失败。`*`= 任意。行压缩条件级联——自上而下读取（第一个匹配项获胜）。

测试1	测试2	测试3	测试4	测试5	决定
F	*	*	*	*	检查点/标准化器故障；请勿运行任何 Gazebo 测试
W	*	*	*	*	首先运行 §10.3 和 §10.4；如果两者都通过，则继续；否则重新培训。
P	F	*	*	*	`end_weight`扭曲路由；降低；重新训练
P	P	F	*	*	使用末端权重消融法重新训练（§10.5）；不要继续
P	P	W	*	*	如果末段 raw_dx ≥ 0.15 但超调 OK → 发货；否则将 `deployment_action_steps`8→4 并重新运行测试 3
P	P	P	F	*	KD树投影失效；请在重新训练之前调试§10.4
P	P	P	W	P	将 0.5提高`obstacle_safe_margin`到 0.6；仅重新运行测试 4
P	P	P	P	F	闭环泛化差距；利用世界多样性增强进行重新训练
P	P	P	P	W	聚类分析失败的世界；如果属于同一拓扑族，则标记已知极限并**进行 SHIP 操作**
P	P	P	P	P	**SHIP** → 冻结 ckpt 作为`phase1a_r2_final.ckpt`；开始阶段 1b

10.诊断工具箱

10.1 训练批次末段实际验证

```python
# verify_end_share.py
import torch, hydra, omegaconf
from hydra.utils import instantiate
cfg = omegaconf.OmegaConf.load("$CKPT_DIR/.hydra/config.yaml")
dataset = instantiate(cfg.task.dataset)
loader = torch.utils.data.DataLoader(dataset, batch_size=256, shuffle=False, num_wor
n_end, n_total = 0, 0
for batch in loader:
    state = batch['obs']['agent_pos']            # (B, n_obs, 9)
    dist = state[:, 0, 6]                         # dist channel (idx 6 in 9-d state
    n_end += (dist < 1.5).sum().item()
    n_total += dist.numel()
    if n_total > 50000:
        break
print(f"末段 (d_goal<1.5m) batch 占比 = {n_end/n_total:.4f}")
print(f"目标 (with end_weight={cfg.task.dataset.end_weight}): "
      f"~{cfg.task.dataset.end_weight * 74 / 53235:.4f}")
```

预期：`end_weight=20`，份额≈20.74/53235≈2.8%（与未加权的0.14%相比）。

10.2 SequenceSampler 索引断言（选项 2 健全性）

```python
# verify_sequence_sampler.py
from diffusion_policy.common.sampler import SequenceSampler
import zarr
buf = zarr.open("/home/cjj/AUV/avoidance_dataset/merged_body_12.zarr", mode='r')
sampler = SequenceSampler(replay_buffer=buf, sequence_length=16,
                          pad_before=2, pad_after=0, episode_mask=None)
for idx in range(0, len(sampler), max(1, len(sampler)//500)):
    sampler.sample_sequence(idx)
    s = sampler.indices[idx]
    assert (s[1] - s[0]) <= 16, f"window too long at {idx}"
    assert s[3] - s[2] == s[1] - s[0], "buffer/sample slice mismatch"
print(f"OK, {len(sampler)} windows verified")
```

10.3 z投影生效验证（修复4后）

在呼叫地点`ml_planner_gazebo.py`附近`_project_trajectory`：

```python
rospy.loginfo_throttle(2.0,
    f"[Z-PROJ] traj_z range = ({z_proj.min():.2f}, {z_proj.max():.2f}) "
    f"clamp_hits = {((z_proj < 0.5) | (z_proj > self.z_max)).sum()}")
```

在 60 秒的飞行过程中，预计会出现以下情况：至少有一条轨迹线，表明`clamp_hits > 0`原始轨迹向地面/天花板方向漂移。如果`clamp_hits == 0`在整个飞行过程中，z 轴投影始终处于无响应状态——请确认`self.z_max=3.0`并应用正确`cumsum`的投影方式。`DPOS_W[2]`

10.4 障碍物投影生效验证

```python
# Inside _project_trajectory after KD-Tree query
rospy.loginfo_throttle(1.0,
    f"[OBS-PROJ] min_dist={d_min:.3f} margin={self.obstacle_safe_margin:.2f} "
    f"repulse_norm={np.linalg.norm(repulse):.3f} "
    f"applied={(d_min < self.obstacle_safe_margin)}")
```

如果在 N=4 次运行期间，`applied=True`无人机与最近障碍物的距离小于 0.5 米（从 rviz 中可见）时没有日志记录，则障碍物投影失效。请重新检查 KD 树是否基于**世界坐标**系点云构建（修复 3），以及是否基于世界坐标系轨迹查询（修复 4）。

10.5 `end_weight` 重力消融

如果测试 3 在 G5（末段 raw_dx）上失败，则重新训练三个短消融周期（每个周期从当前 ckpt 作为热启动，而不是从头开始）：

狂欢

```
for EW in 10 20 30; do
    python train.py --config-name=phase1a_r2 \
        task.dataset.end_weight=$EW \
        training.max_epochs=10 \
        training.resume_path=$CKPT_DIR/checkpoints/latest.ckpt \
        hydra.run.dir=data/outputs/phase1a_r2_ew${EW} &
done
wait
# Eval Test 3 (N=4 only) on each; pick the smallest end_weight where
# 末段 raw_dx < 0.15 AND detour < 1.5. Larger end_weight risks Test-2 regression.
```

---

11. 时间预算

阶段	挂钟	笔记
§2 领先	10分钟	主要使用 grep + sanity
§4 测试 1 离线	15分钟	GPU 瓶颈
§5 测试 2 N=2	运行 5 分钟 + 分析 2 分钟	1 次试验
§6 测试 3 N=4 ×5	5 × (3 分钟运行 + 30 秒重置) ≈ 20 分钟 + 5 分钟分析	核心测试
§7 测试 4 多	5 × (3.5 分钟运行 + 30 秒重置) ≈ 20 分钟 + 5 分钟分析	
§8 测试 5 已关闭20	20 × (3.5 分钟运行 + 30 秒重置) ≈ 80 分钟 + 10 分钟分析	最长的一步
§9–10 诊断（如有）	60分钟（可变）	仅当警告/失败时
缓冲 + 报告撰写	60分钟	
全部的	**约 5 小时纯运行 + 1 小时分析 + 1 小时诊断 + 1 小时报告 ≈ 8 小时**	一个工作日

---

12. 附录——`analyze_n4_test.py`（完整、可执行）

保存到`~/ml_planner_log/scripts/analyze_n4_test.py`。

Python

```python
#!/usr/bin/env python3
"""Phase 1a R2 — N=2 / N=4 / multi / closed20 analyzer."""
import argparse, glob, os, json
from pathlib import Path
import numpy as np
import pandas as pd

GOAL_TOL = 0.5

def find_pair(run_dir):
    """Return (pos_cmd_csv, diffusion_csv) inside a single run directory."""
    pos = sorted(glob.glob(f"{run_dir}/pos_cmd_*.csv"))
    dif = sorted(glob.glob(f"{run_dir}/diffusion_*.csv"))
    return (pos[-1] if pos else None, dif[-1] if dif else None)

def analyse_run(run_dir, goal_tol=GOAL_TOL):
    pos_csv, dif_csv = find_pair(run_dir)
    if pos_csv is None:
        return {"run": run_dir, "error": "no pos_cmd csv"}
    pos = pd.read_csv(pos_csv)
    # required: stamp, odom_x, odom_y, odom_z, goal_x, goal_y, goal_z
    # optional: min_clr
    dist = np.sqrt((pos['odom_x'] - pos['goal_x'])**2 +
                   (pos['odom_y'] - pos['goal_y'])**2)
    path_len = float(np.sum(np.sqrt(np.diff(pos['odom_x'])**2 +
                                    np.diff(pos['odom_y'])**2 +
                                    np.diff(pos['odom_z'])**2)))
    disp = float(np.sqrt((pos['odom_x'].iloc[-1] - pos['odom_x'].iloc[0])**2 +
                         (pos['odom_y'].iloc[-1] - pos['odom_y'].iloc[0])**2))
    detour = path_len / max(disp, 1e-3)
    duration = float(pos['stamp'].iloc[-1] - pos['stamp'].iloc[0])
    final_d = float(dist.iloc[-1])
    success = bool(final_d < goal_tol)

    # max_overshoot
    if (dist.values < 1.0).any():
        enter_idx = int(np.argmax(dist.values < 1.0))
        post = dist.values[enter_idx:]
        max_over = float(max(0.0, post.max() - dist.values[enter_idx]))
    else:
        enter_idx = -1
        max_over = float('inf')

    # 末段 raw_dx
    end_dx_mean = float('nan')
    if dif_csv:
        dif = pd.read_csv(dif_csv)
        merged = pd.merge_asof(
            dif.sort_values('stamp'),
            pos[['stamp','odom_x','odom_y','goal_x','goal_y']].sort_values('stamp'),
            on='stamp', direction='nearest', tolerance=0.5)
        merged['d'] = np.sqrt((merged['odom_x'] - merged['goal_x'])**2 +
                              (merged['odom_y'] - merged['goal_y'])**2)
        mask = merged['d'] < 1.0
        if mask.any():
            end_dx_mean = float(np.abs(merged.loc[mask, 'raw_dx']).mean())

    # clearance
    if 'min_clr' in pos.columns:
        clr = pos['min_clr'].dropna().values
        share05 = float((clr < 0.5).mean()) if len(clr) else None
        n02 = int((clr < 0.2).sum()) if len(clr) else None
    else:
        share05, n02 = None, None

    # duty_traj — fraction of ticks where pos_cmd changed (proxy for non-stale)
    if 'cmd_x' in pos.columns:
        diffs = np.diff(pos['cmd_x'].values) ** 2 + np.diff(pos['cmd_y'].values) **
        duty = float((diffs > 1e-8).mean())
    else:
```

```
        duty = None

    return dict(run=Path(run_dir).name, success=success, path_len=path_len,
                disp=disp, detour=detour, duration=duration, final_d=final_d,
                max_overshoot=max_over, end_dx_mean=end_dx_mean,
                share_clr05=share05, n_clr02=n02, duty_traj=duty)

def aggregate(rows, mode):
    df = pd.DataFrame(rows)
    print(df.to_string(index=False))
    out = {"mode": mode, "rows": rows}
    if mode == "n4_x5":
        succ = int(df['success'].sum())
        out["pass_G3"] = bool(df['detour'].mean() < 1.5)
        out["pass_G4"] = bool(df['max_overshoot'].mean() <= 1.0)
        out["pass_G5"] = bool(df['end_dx_mean'].mean() < 0.15)
        out["pass_succ"] = bool(succ >= 4)
        print(f"\n[N=4 ×5] success={succ}/5, "
              f"mean detour={df['detour'].mean():.2f}, "
              f"mean max_overshoot={df['max_overshoot'].mean():.2f} m, "
              f"mean end_dx={df['end_dx_mean'].mean():.3f} m")
    elif mode == "closed20":
        succ = int(df['success'].sum())
        out["pass_G7"] = bool(succ / max(len(df), 1) >= 0.70)
        print(f"\n[closed20] success={succ}/{len(df)}")
    elif mode == "n2":
        ok = bool(df['success'].all() and df['detour'].mean() <= 1.40
                  and df['duration'].mean() <= 140)
        out["pass_G2"] = ok
    elif mode == "clr":
        # aggregate across runs
        c05 = df['share_clr05'].dropna().mean()
        n02 = int(df['n_clr02'].fillna(0).sum())
        out["pass_G6"] = bool((c05 is not None) and (c05 < 0.015) and (n02 == 0))
        print(f"\n[multi clr] mean share<0.5={c05}, total frames<0.2={n02}")
    return out

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--run_dir", required=True)
    ap.add_argument("--mode", required=True, choices=["n2","n4_x5","clr","closed20"])
    ap.add_argument("--goal_tol", type=float, default=0.5)
    ap.add_argument("--baseline_path", type=float, default=None)
    ap.add_argument("--baseline_disp", type=float, default=None)
    ap.add_argument("--baseline_detour", type=float, default=None)
    ap.add_argument("--baseline_dur", type=float, default=None)
    ap.add_argument("--baseline_max_overshoot", type=float, default=None)
    ap.add_argument("--report", default=None)
    args = ap.parse_args()
    sub = sorted([d for d in glob.glob(f"{args.run_dir}/*") if os.path.isdir(d)])
    rows = [analyse_run(d, args.goal_tol) for d in sub]
    out = aggregate(rows, args.mode)
    if args.report:
        Path(args.report).write_text(json.dumps(out, indent=2, default=str))
        print(f"report → {args.report}")

if __name__ == "__main__":
    main()
```

==========

文件 2 — `/mnt/user-data/outputs/Phase1b_Gym_Wrapper_Design.md`

==========

第一阶段 b — 健身房包装设计（无人机避障强化学习基础设施）

所有者：Scarlettfw / iDP3-UAV **阶段目标**：4 周构建完成`DroneAvoidanceEnv`，并通过在微不足道的奖励上进行 1 小时的 PPO 学习进行验证。 **取决于**：阶段 1a R2 检查点已被接受`Phase1a_R2_Validation_Protocol.md`。

1.目的与定位

【事实】DPPO（Ren 等人，ICLR 2025，arxiv 2409.00588）证明，使用行为克隆预训练的扩散策略可以通过 PPO 在两层“扩散 MDP”上进行微调。论文原文指出：“*据我们所知，DPPO 是第一个能够以高成功率（>50%）从状态或像素输入解决运输问题的强化学习算法*”（机器人模拟运输）。在家具长凳*单腿任务*上，论文原文指出：“*值得注意的是，DPPO 将实际性能提升至 80%（20 次试验中成功 16 次）。虽然高斯策略在微调后在仿真中取得了较高的成功率（88%），但在硬件上却完全失败（0%）。*”*这验证了*第二阶段的**路线 A**：采用第一阶段 A 的 R2（行为克隆预训练），并在 Gazebo 中使用 DPPO 进行微调。arxiv +3 + 3

第 1b 阶段的目的**仅在于基础设施——**构建一个与 Gymnasium 兼容的环境，该环境封装了现有的第 1a 阶段 R2 推理循环 + Gazebo/PX4 堆栈，以便 DPPO（或 PPO 基线）可以插入，而不会对训练器进程造成 ROS 污染。

【裁决】第一阶段b期发货时间：

- 一个 60 秒的随机策略回合可以端到端运行（`reset → step×N → done`），没有 Gazebo 死锁。
- 通过加载的第 1a 阶段 R2 检查点`Phase1aPolicyAdapter`在环境内的 20 集推广中得分 ≥ 60%（而原始闭环约为 70%——10% 的开销是可以容忍的）。
- 具有简单高斯头部的 PPO 基线在 Phase 1a 均值上学会了在 1 小时实际时间内*不会立即崩溃（积极的奖励趋势，而不是绝对性能）。*

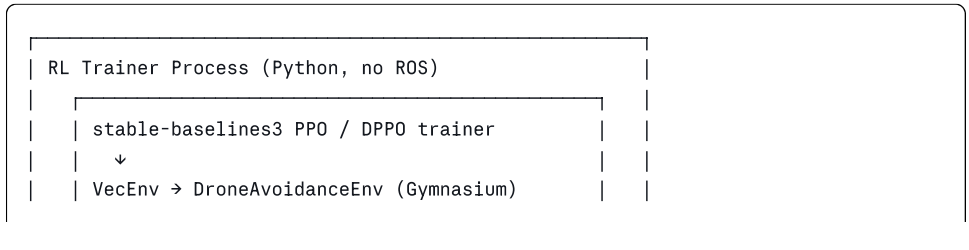
---

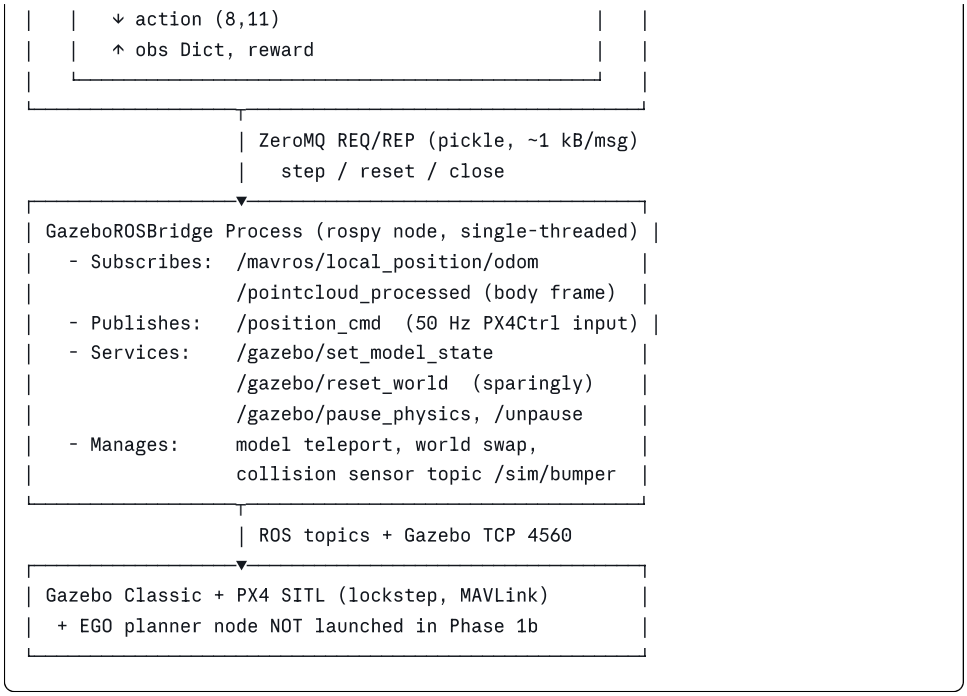
2. 设计原则

原则	具体含义
体育馆标准	`gymnasium.Env`，返回`reset(seed, options)``step``(obs, reward, terminated, truncated, info)`稳定基线3
第一阶段a 热启动保留	动作空间 = 轨迹级别 (8×11)，因此预训练的扩散头可以直接插入而无需任何改动。
奖励模块化和加权	`RewardComposer`每个项的权重均可通过 Hydra 配置进行更改；**环境代码中未内置奖励塑造机制。**
ROS/Gazebo 隔离	所有 ROS 代码都位于`GazeboROSBridge`（一个独立的 rospy 进程中）；训练器仅导入纯 Python 代码`DroneAvoidanceEnv`，该代码通过 ZeroMQ 与桥接器通信。
无头	`HEADLESS=1`强制性；目标是`PX4_SIM_SPEED_FACTOR=2`在稳定性验证之后进行。[事实——PX4 官方文档，sim_gazebo_gz：“*模拟器同步运行，这意味着 Gazebo 运行模拟器的速度与 PX4 的速度相同……*`GitHub`*性能强大的台式机通常可以以 6-10 倍的速度运行模拟程序，而笔记本电脑的运行速度约为 3-4 倍。*”PX4自动驾驶仪 GitHub在第一阶段 b 中，我们将上限设为 2 倍，以保持同步超时裕度。
确定性种子	种子路由至 (a)`WorldSampler`随机选择，(b) 无人机初始化偏移量，(c) 策略噪声
EGO规划器完全被绕过	第一阶段 1b 仅使用第一阶段 1a 的 R2 + RL；不启动 EGO 节点。这是有意为之的设计决策——在第二阶段 2c 中，将 EGO 作为“安全回退”方案重新添加。

---

3. 架构图





---

4. 观测空间

【事实】第一阶段a的状态是……`9d = pos_world(3) + goal_body(3) + dist(1) + sin_yaw(1) + cos_yaw(1)`点云是`(4096, 6)`……`n_obs_steps = 3`

```python
import gymnasium as gym
import numpy as np

self.observation_space = gym.spaces.Dict({
    "agent_pos":   gym.spaces.Box(low=-np.inf, high=np.inf, shape=(3, 9),       dtyp
    "point_cloud": gym.spaces.Box(low=-np.inf, high=np.inf, shape=(3, 4096, 6), dtyp
})
```

在环境内部，通过一个填充框来维护三步窗口`collections.deque(maxlen=3)`，当可用观测值少于三个时，填充框会用第一个观测值填充（与阶段 1a 的部署行为一致）。[假设] 阶段 1b 逐字节地保持 9 天状态和 PC 通道定义不变。任何偏差都会导致热启动失败。

---

5. 动作空间——核心设计决策

选项 A — 轨迹级别 (8×11)（推荐）

```python
self.action_space = gym.spaces.Box(low=-1.0, high=1.0, shape=(8, 11), dtype=np.float
```

- 匹配阶段 1a`n_action_steps = 8`，动作暗度 11（`dpos_world(3) + dvel_world(3) + acc_world(3) + dyaw(1) + yaw_rate(1)`）。
- 与预训练的扩散头兼容，可作为演员。
- [事实 — DPPO GitHub README，irom-princeton/dppo，逐字稿]："`horizon_steps`：*预测动作块大小（应该与执行动作块大小相同`act_steps`，使用 MLP。使用 UNet 时可能不同，例如，`horizon_steps=16`和`act_steps=8`）*"。GitHub
- 【事实——DPPO论文§4.3，原文】：*"我们发现，DPPO受益于使用较大的T_p进行预训练（预测效果更好），以及使用较小的T_a进行微调（更易于策略梯度调整）"*，脚注为：*"对于MLP中的全连接层，我们通过实验发现，使用不同的块大小进行MLP的预训练和微调会导致训练不稳定。"*第一阶段a使用MLP风格的头部`T_p=16, T_a=8`——如果我们想在微调期间缩小T_a，我们应该切换到UNet头部，或者接受`T_a = T_p = 8`微调。**未解决的问题【待定】——已标记为需要Zread处理。** arXiv arXiv

方案 B — 低级别（每步 11 天）

- 更清晰的PPO理论；标准高斯头部。
- **打破第一阶段热启动——**需要从头开始训练或提炼，失去先前的BC。

决定

选择方案 A。权衡取舍已接受：PPO 基线需要 88 天的行动期（比 11 天更难），但从第一阶段 R2 开始的快速启动对于项目进度而言是不可协商的。在第二阶段（建立 DPPO 基线之后）回退到方案 B 需要 1-2 周的重新培训；提前回退则会使第一阶段的全部投资付诸东流。

【待定】是否在执行下一个步骤之前执行全部 8 个子操作`env.step`（1 个环境步骤 = 8 × 0.3 秒 = 2.4 秒模拟时间），还是仅执行 4 个子操作（1.2 秒）`deployment_action_steps`。**默认值为 4 个**（与第一阶段 R2 部署相匹配）。将在首次 PPO 学习曲线完成后重新评估。

---

6. 步骤语义

```python
def step(self, action: np.ndarray):
    """action: (8, 11) in normalized [-1, 1]."""
    assert action.shape == (8, 11), action.shape

    # 1. De-normalize via Phase 1a normalizer
    raw_action = self.normalizer['action'].unnormalize(action)   # (8, 11)

    # 2. Execute first `deployment_action_steps` sub-actions
    n_exec     = self.deployment_action_steps                    # default 4
    cur_pos_w  = self._latest_pos_world                           # from bridge
    cur_yaw    = self._latest_yaw

    for k in range(n_exec):
        # cumsum dpos_world -> world-frame setpoint
        cur_pos_w = cur_pos_w + raw_action[k, 0:3]                # DPOS_W
        cur_yaw   = cur_yaw   + raw_action[k, 9]                  # DYAW
        self.bridge.send_pos_cmd(cur_pos_w, cur_yaw,
                                 vel=raw_action[k, 3:6],
                                 acc=raw_action[k, 6:9])
        self.bridge.wait_dt(0.3)   # uses sim clock under lockstep

    # 3. Next obs (3-step sliding window updated by bridge)
    next_obs = self.bridge.get_obs_window()

    # 4. Reward
    info = self.bridge.get_info()                # min_clr, did_collide, did_arrive,
    reward, breakdown = self.reward_composer(next_obs, info,
                                             prev_dist=self._prev_dist,
                                             raw_action=raw_action[:n_exec])
    self._prev_dist = info['dist']

    terminated = bool(info['did_arrive'] or info['did_collide'] or info.get('oob', F
    truncated  = bool(self._step_count >= self.max_steps)
    self._step_count += 1
    info.update({"reward_breakdown": breakdown})
    return next_obs, reward, terminated, truncated, info
```

锁定步进与实时

【不确定】第一阶段 b 以默认的锁定步长 (1×) 开始，以验证确定性，然后`PX4_SIM_SPEED_FACTOR=2`在 100 轮稳定性测试后推进。【事实 — PX4 文档】“Gazebo [Ignition] 无法禁用锁定步长”—`wait_dt(0.3)`当 PX4 完成模拟时，模拟时间会精确推进。更高的锁定步长可能导致 MAVLink 超时 → 第一阶段 b 的锁定步长上限为 2×。GitHub

---

7. 重置语义

```python
```

```python
def reset(self, seed=None, options=None):
    super().reset(seed=seed)
    rng = np.random.default_rng(seed)

    # 1. World choice — 60 stratified worlds by obstacle density
    world_path = self.world_sampler.choose(rng)
    self.bridge.swap_world(world_path)

    # 2. Drone teleport — start (0,0,2) ± 0.5 m
    start = np.array([0.0, 0.0, 2.0]) + rng.uniform(-0.5, 0.5, size=3)
    start[2] = np.clip(start[2], 1.5, 2.5)
    self.bridge.teleport_drone(start, yaw=0.0)

    # 3. Goal sampling — d in [6, 10] m, within current world bounds
    goal = self.world_sampler.sample_goal(world_path, rng, d_range=(6.0, 10.0))
    self.current_goal = goal

    # 4. Hover stabilization
    self.bridge.unpause()
    self.bridge.wait_dt(3.0)

    # 5. PX4 → CMD_CTRL (offboard with pos_cmd)
    self.bridge.set_mode("CMD_CTRL")

    # 6. Reset obs-window deque + counters
    self.bridge.reset_obs_window()
    obs = self.bridge.get_obs_window()
    self._prev_dist = float(np.linalg.norm(start[:2] - goal[:2]))
    self._step_count = 0
    return obs, {"world": world_path, "goal": goal.tolist()}
```

【判定】EGO规划器**未在Phase 1b中启动**。启动文件与Phase 1a`single_AUV_phase1b.launch`有所不同，`single_AUV.launch`移除了`<node name="ego_planner_node" .../>`相关模块。设计决策**D1**：Phase 1b中的所有碰撞规避均来自Phase 1a的R2头部以及强化学习的微调。

【假设】`/gazebo/reset_world`可能会留下残留的物理状态——问题 #311 中记录了一个稳定性问题，即在多次调用后`ethz-asl/rotors_simulator`"四旋翼飞行器的旋翼消失，四旋翼飞行器无法起飞"。阶段 1b 使用**世界交换+ 选择性实体重置**来代替。如果出现不稳定情况，则回退到每个回合的完整操作（每次重置约 30 秒）。**`set_model_state`** `roslaunch` GitHub

---

8. 奖励设计

8.1 奖励条款

学期	定义	初始重量	阶段
`r_progress`	`prev_dist - cur_dist`（每环境步长Δ米）	+1.0	1b.1
`r_arrival`	`+R_arr if cur_dist < 0.5 else 0`, `R_arr = 10`	+10.0	1b.1
`r_clearance`	`-c_pen if min_clr < 0.5`此外，`-100`碰撞时也会发生这种情况。	安全系数 1.0 / 碰撞系数 100.0	1b.2
`r_smoothness`	`-‖raw_action[k+1, 0:3] - raw_action[k, 0:3]‖²`已执行子动作的平均值	0.05	1b.3
`r_time`	`-0.01`每个环境步骤	0.01	1b.1
`r_z`	`−max(0,	z − 2.0	− 1.0)`（z ∈ [1, 3] 之外的惩罚）

全部的：`r = w_p·r_prog + w_a·r_arr − w_c·r_clr − w_t·r_time − w_s·r_smooth − w_z·r_z`。

8.2 分阶段训练

次级阶段	奖励条款	目的	成功标准
1b.1	进度 + 到达 + 时间	使用随机初始化和 PPO 验证环境的可学习性	100k 环境步骤 → 平均回合收益在 5 次评估中单调递增
1b.2	+ 清空 + z	增加安全性。从 1a R2 电路重新热启动	clr<0.5 速率≤阶段1a闭合20基线收敛
1b.3	+ 平滑度	最后抛光	平均每块 Δ_dpos 抖动减半（与 1b.2 端点相比）

8.3 奖励等级校准

[UNCERTAIN] PopArt (Hessel et al. 2018, arxiv 1809.04474) automatically normalizes returns via running mean/var on the value head; stable-baselines3 PPO does not ship PopArt by default. **Recommended path for Phase 1b**: manual calibration via a 500-step random-policy rollout. Compute per-term running mean of `|r_*|` and choose weights so each enabled term contributes within one order of magnitude (`w_i · E[|r_i|]` in `[0.1, 10.0]` for each `i`). Re-calibrate when 1b.2 / 1b.3 add terms. Flag for Zread.

---

9. Termination & Truncation

Condition	terminated	truncated	info flag
`dist < goal_tolerance (0.5 m)`	True	False	`arrived=True`
Collision (bumper plugin)	True	False	`collided=True`
OOB (`	x	>15`or`	y
Step count ≥ `max_steps` (default 80 ≈ 2.4 s × 80 = 3.2 min sim)	False	True	`timeout=True`
No progress (Δ_dist < 0.05 m over last 20 steps)	False	True	`stuck=True`

---

10. Phase 1a checkpoint warm-start

```python
```

```python
# phase1a_adapter.py — skeleton
import torch
from omegaconf import OmegaConf
from hydra.utils import instantiate

class Phase1aPolicyAdapter:
    """
    Wraps a Phase 1a R2 iDP3 checkpoint so it can act as the policy in a
    stable-baselines3 / DPPO trainer.
    Exposes: predict(obs_dict, deterministic) -> (action, state)
             log_prob(obs_dict, action)       -> tensor   (needed for PPO ratio)
    """
    def __init__(self, ckpt_path: str, device: str = "cuda:0"):
        ckpt = torch.load(ckpt_path, map_location=device)
        cfg = OmegaConf.create(ckpt['cfg'])
        self.policy = instantiate(cfg.policy)
        self.policy.load_state_dict(ckpt['state_dicts']['model'])
        self.policy.to(device).eval()
        self.normalizer       = self.policy.normalizer
        self.device           = device
        self.n_obs_steps      = cfg.n_obs_steps       # 3
        self.n_action_steps   = cfg.n_action_steps    # 8
        self.horizon          = cfg.horizon           # 16

    def predict(self, obs_dict, deterministic=False):
        # World-frame raw obs — Fix 4 ensures policy normalizes internally.
        with torch.no_grad():
            out = self.policy.predict_action(
                {k: torch.from_numpy(v).to(self.device).unsqueeze(0)
                 for k, v in obs_dict.items()},
                deterministic=deterministic,
            )
        # out['action']: (1, n_action_steps, 11), NORMALIZED.
        return out['action'].squeeze(0).cpu().numpy(), None

    def log_prob(self, obs_dict, action):
        """
        For DPPO: log-prob of (8, 11) action chunk under the denoising chain.
        For PPO baseline (Phase 1b.1), unused — instead wrap a Gaussian head
        over predict's mean with fixed σ=0.05 in normalized action space.
        """
        raise NotImplementedError("Wire when DPPO branch lands (Phase 2, W1)")
```

SB3 integration (1b.1 sanity)

Wrap a Gaussian (μ from `predict`, σ=0.05) around the deterministic Phase 1a output. This suffices to learn `r_progress + r_arrival`. Real DPPO log-prob arrives at Phase 2 (out of scope here).

[FACT — DPPO paper §4.3 verbatim]: *"Instead of fine-tuning the pre-trained model weights θ, we make a copy θ_ft — θ is frozen and used for the early denoising steps, while θ_ft is used for the last K' steps and updated with DPPO."* [FACT — DPPO README verbatim]: *"In our Furniture-Bench experiments we use* `denoising_steps=100`, `model.ddim_steps=5`, *and* `ft_denoising_steps=5`." For Phase 1a R2 (DDPM with ~100 steps in pre-training), the Phase 2 plan should pre-record `ddim_steps=5, ft_denoising_steps=5` as the starting configuration. [FACT — DPPO paper §4.3 verbatim]: *"clipping σ_k to a higher minimum value (denoted σ^exp_min, e.g., 0.01–0.1) when sampling actions helps exploration."* arxiv + 3

---

11. Implementation Timeline (4 weeks)

Week	Tasks	Validation product
W1	`GazeboROSBridge` rospy node + ZeroMQ bridge; teleport / reset utilities; bumper plugin on drone URDF	100 reset cycles without rotor-disappearance bug; `step()` returns valid obs within 0.4 s sim

Week	Tasks	Validation product
W2	`DroneAvoidanceEnv` with §4–§9 wired; `WorldSampler` over 60 worlds; reward composer for 1b.1 terms	`gymnasium.utils.env_checker.check_env(env)` passes; 50-episode random-policy trace dumped
W3	`Phase1aPolicyAdapter` + SB3 `PPO("MultiInputPolicy", env)` baseline on 1b.1 reward	100 k env steps PPO: mean ep return non-decreasing; Phase 1a ckpt scored ≥ 60 % under env
W4	1b.2 reward terms + collision detection; reward-scale calibration; documentation + Zread submission for Phase 2 DPPO wiring	5 h training shows clr<0.5 share drop vs W3 baseline

12. 依赖检查清单

Dependency	Status check	Owne
Phase 1a R2 ckpt passing `Phase1a_R2_Validation_Protocol.md`	required	Scarle
World generator producing 60 stratified worlds (n_obs ∈ {2,4,6,8,10,12})	exists at `~/AUV/catkin_ws0/.../worlds/generator/`; needs `--stratify` flag	Scarle
EGO planner can be removed from launchfile cleanly (no dangling subscriber)	grep PX4Ctrl subs; stub any `/ego/...` expectation	W1
PX4Ctrl `/position_cmd` topic stays open after bridge takes over	confirm in current `single_AUV.launch`	W1
Gazebo bumper plugin / collision topic	[UNCERTAIN] — see §13 Q1	W1
ZeroMQ Python deps in trainer env	`pip install pyzmq`	W1

13. Open Questions & [UNCERTAIN]

Q1 — Collision detection data source [HIGH PRIORITY]

Option	Pros	Cons	Verdict
Gazebo `libgazebo_ros_bumper.so` on drone base_link	Native; deterministic; cheap	Documented URDF→SDF collision-name suffixing pitfall (ROS Answers question 246448; iRobot Create 2 simulation blog: "when the URDF file is converted to a SDF file, a collision name is generated and referenced by the sensor plugin"), Natesh's Dev Blog! often causing empty `states[]` arrays	**Recommended.** Validate W day 1: `gz sdf -p <urdf> /tmp/robot.sdf` and gre for the auto-suffixed collisi name.

Option	Pros	Cons	Verdict
IMU spike (`‖acc_world‖ > threshold`)	No plugin work	Environment-dependent thresholds; can miss soft contacts	Backup.
Polling `min_clr < 0.0` from world-frame PC + obstacle positions	Already computed in Fix 3's `_obstacle_project`	Sparse PC on contact side may miss collisions	Cross-check signal (`info["min_clr_floor'`

Plan: bumper as primary, polling as cross-check.

Q2 — `deployment_action_steps` after RL fine-tune (4 vs 8)

Phase 1a R2 uses 4. [FACT — DPPO paper §4.3]: smaller `T_a` is more amenable to policy gradient. **Decision deferred to Phase 2 ablation.** Phase 1b ships with `deployment_action_steps = 4`.

Q3 — Reward scale calibration: PopArt vs manual

Manual at Phase 1b suffices (single env). PopArt becomes useful at Phase 2 multi-env mixes. Defer.

Q4 — PPO baseline vs jumping straight to DPPO

[JUDGMENT] **Do PPO first.** SB3 PPO with a Gaussian head (μ from Phase 1a, fixed σ=0.05) is enough to validate the env, reset, reward composition, and Phase 1a integration. Going straight to DPPO conflates "is the env right?" with "is the diffusion-MDP log-prob right?". DPPO comes in Phase 2 W1.

Q5 — Action dim reduction (11 → 4: dpos_world + dyaw only)

[JUDGMENT] Tempting (88→32 action) but **rejected for Phase 1b**: breaks Phase 1a head's output shape. If post-RL diagnostics show velocity/acc/yaw-rate dims unused, revisit at Phase 2c.

Q6 — Policy head MLP vs UNet for fine-tuning

[UNCERTAIN — high impact] [FACT — DPPO README verbatim]: *"With fully-connected layers in MLP, empirically we find that using different chunk sizes for pre-training and fine-tuning with MLP leads to training instability."* If Phase 1a R2 used MLP head with `T_p=16, T_a=8`, we cannot freely shrink `T_a` during fine-tuning. Two outs:

- (a) re-pretrain a small UNet head from current ckpt encoder, with `T_p=16`;
- (b) accept `T_a = T_p = 8` for fine-tuning (simpler). **Flag for Zread to verify against current Phase 1a config.**

Q7 — Step time budget under `PX4_SIM_SPEED_FACTOR=2`

[UNCERTAIN] At 2×, the 1.2 s sim chunk → 0.6 s wall clock. Bridge ZeroMQ round-trip + obs collection target: < 100 ms. Stay under 3× to keep margin.

---

14. 风险评估表

Risk	Likelihood	Impact	Mitigation
Gazebo `/gazebo/reset_world` instability (rotor disappearance after N resets, per rotors_simulator #311)	Medium-High	High — kills training	Use teleport + selective entity reset; hard fallback to full `roslaunch` per episode if it fails by W2
Phase 1a warm-start does not converge under PPO	Medium	High	Lower σ to 0.02 in normalized action; increase warm-up steps before policy updates; final fallback Option B (Phase 1b restart)
Reward design requires many iterations	High	Medium	Staged 1b.1 → 1b.2 → 1b.3 with explicit rollback gates

Risk	Likelihood	Impact	Mitigation
Training too slow (Gazebo not headless-enough at 2×)	Medium	Medium	Profile in W3; if < 10 k env steps/h, VecEnv with 4 Gazebo instances on separate ports
Bumper plugin returns empty contacts (the URDF→SDF naming bug)	High (well documented)	Low (caught at W1)	Pre-test W1 day 1 with synthetic obstacle drop
DPPO log-prob implementation diverges from `predict_action` (Phase 2 risk)	Medium	High (Phase 2)	Out of scope for 1b; flagged in Phase 2 Zread submission

---

15. Zread Submission Preparation Note

The companion file `Zread_Submission_Phase1b_Design.md` lists the design choices in this document that require iDP3 source-code cross-checks (warm-start adapter shape compatibility, normalizer reuse, `n_obs_steps`/`n_action_steps` semantics, MLP vs UNet head). Submit after this design is reviewed and committed.

==========

FILE 3 — `/mnt/user-data/outputs/Zread_Submission_Phase1a_R2_Validation.md`

==========

Zread Submission — Phase 1a R2 Validation

Submission date: TBD (after Phase 1a R2 Validation Protocol completes) **Author**: Scarlettfw
Companion artifact: `Phase1a_R2_Validation_Protocol.md` + the resulting `~/ml_planner_log/<RUN_TAG>/closed/report.md`

---

1. 提交背景与上下文

Phase 1a R2 introduces six changes to the Phase 1a baseline iDP3-UAV pipeline:

- **Fix 3**: `_obstacle_project` converts body-frame point cloud → world-frame (`Rz(yaw) @ pc_body + cur_pos`) before building the KD-Tree.
- **Fix 4**: `predict_action` uses `obs_dict` (world-frame raw) for the KD-Tree, not the internally normalized `nobs`.
- **Opt 1**: PCA removed; obstacle repulsion direction is `(trajectory_point - nearest_obstacle)` — saves 1–2 s CPU per inference.
- **Opt 2**: `SequenceSampler.indices` augmented with consistency assertions.
- **末段加权采样**: training dataset's `_compute_sample_weights` upweights windows whose start frame has `d_goal < 1.5 m`. `end_weight ∈ {10, 20, 30}` candidates.
- **KD-Tree obstacle repulsion**: integrated into `_project_trajectory` with `obstacle_safe_margin = 0.5 m`.

User-verified data context (the basis for end-weighting):

- 60 151 frames total across 988 episodes (76 % success rate).
- 6.22 % of frames (3 743) have `d_goal < 1 m`.
- Only **74** of those survive the `t + n_action_steps ≤ ep_len` window-start constraint.
- Those 74 are **0.14 %** of 53 235 legal windows → unweighted, the model sees one 末段 sample per ~700 batches.
- Deceleration signal is present in data (Δx_mean = 0.012 at `d<1m` vs 0.033 at `d≥4m`) but is not being learned.
- Old Phase 1a failure mode: at `d_goal = 0.79 m`, `raw_dx_w = 0.293 m` (saturating the `±1.5/8 ≈ 0.1875` normalizer upper bound) — the model **does not decelerate**.

- z position: mean 1.97 m, p95 2.66 m, max 4.15 m; only 0.1 % of frames have `z > 3.0 m`. Deployment `z_max=3.0` vs policy-side `z_max=5.0` mismatch flagged.

The validation protocol attached defines closing criteria. This submission asks Zread to review **the protocol's gate thresholds and diagnostic decisions** *before* running it on a fresh R2 ckpt.

---

2. Items requiring Zread review

#	Item	Type	Location
R1	Gate **G3** threshold: mean detour < 1.5 in N=4 ×5	Threshold	Protocol §1, §6.4
R2	Gate **G5** threshold: 末段 `raw_dx < 0.15 m` (vs 0.293 baseline). Is 0.15 sufficient evidence of deceleration, or should it be < 0.1?	Threshold	Protocol §6.4
R3	`max_overshoot` definition: first crossing of `dist<1.0 m`, max thereafter, threshold 1.0 m	Algorithm	Protocol §6.3
R4	Decision-matrix row "W in offline, P elsewhere → accept after §10.3+§10.4 verification" — too permissive?	Policy	Protocol §9
R5	`end_weight` ablation plan: train 3 short fine-tunes from R2 ckpt as warm-start (10 epochs each), not from scratch	Methodology	Protocol §10.5
R6	Diagnostic §10.3 — z-proj clamp logging — `clamp_hits > 0 over 60 s flight` as "is alive" criterion. Does the reference `ml_planner_gazebo.py` have a similar hook?	Verification	Protocol §10.3
R7	Driver script `analyze_n4_test.py` — esp. the `pd.merge_asof` join between `pos_cmd` and `diffusion` CSVs on `stamp` with `tolerance=0.5 s`. Is stamp consistency guaranteed by Phase 1a logger?	Implementation	Protocol §12
R8	Closing **all** G1–G7 as a hard AND — is any gate redundant given the others?	Logic	Protocol §1

---

3. 已知风险与不确定性 (from [UNCERTAIN] markers)

- **R-U1**: `end_weight` value not yet fixed. R2 was trained with one of {10, 20, 30}. The ablation plan (§10.5) is the contingency.
- **R-U2**: `deployment_action_steps` candidate set {2, 4, 8} not yet pinned. Protocol uses whatever R2 was trained against (recorded in `.hydra/config.yaml`); diagnostic table allows reducing 8 → 4 if overshoot persists.
- **R-U3**: Whether old Phase 1a tests used `goal_tolerance = 0.5 m` consistently is asserted from user data; Zread should sanity-check against the project's standing test protocol if one exists.
- **R-U4**: `obstacle_safe_margin = 0.5 m` is the design candidate; protocol allows 0.5 → 0.6 raise if Test 4 fails on G6 without retraining.

---

4. Verdict requested

Per item R1–R8, please return one of:

- **MUST_MODIFY** — change before running the protocol
- **SUGGEST_MODIFY** — flagged but protocol may proceed
- **OPTIONAL_MODIFY** — nice-to-have
- **PASS** — proceed as-is

Per-item rationale (1–3 sentences). If MUST_MODIFY, provide the specific replacement (number / formula / code change).

[FACT] iDP3 (Ze et al., arxiv 2410.10803) uses *egocentric* 3D representations (camera frame, not world frame) on the Fourier GR1 humanoid with the RealSense L515 LiDAR camera. Our project converts the body-frame point cloud to world frame *before* the KD-Tree (Fix 3). This is a deliberate adaptation for the auxiliary obstacle-projection path, not a change to the network's input frame. Cross-check requests:

- **CC1**: In `YanjieZe/Improved-3D-Diffusion-Policy`, is the point cloud ever converted out of camera frame before reaching the policy network? Confirm our world-frame conversion is **only** for the obstacle-projection auxiliary path, never reaching the network input.
- **CC2**: Verify `SequenceSampler` (from `diffusion_policy.common.sampler`) supports `pad_after=0` cleanly for our `n_action_steps=8` future-required windowing.
- **CC3**: Confirm `predict_action`'s signature and that returning a `(B, n_action_steps, action_dim)` slice (not the full `(B, horizon, action_dim)`) is consistent with both train-time and the deployment loop's expectation.

==========

FILE 4 — `/mnt/user-data/outputs/Zread_Submission_Phase1b_Design.md`

==========

Zread Submission — Phase 1b Gym Wrapper Design

Submission date: TBD (after Phase 1a R2 ships and the Phase 1b design doc is committed)
Author: Scarlettfw **Companion artifact**: `Phase1b_Gym_Wrapper_Design.md`

---

1. 提交背景与上下文

Phase 1b is the RL infrastructure layer for **Phase 2 Route A: DPPO** ([FACT] Ren et al., ICLR 2025, arxiv 2409.00588). Concrete DPPO evidence motivating Route A:

- [FACT — DPPO paper §5.4 verbatim]: *"Notably, DPPO improves the real-world performance to 80% (16 out of 20 trials). Though the Gaussian policy achieves a high success rate in simulation after fine-tuning (88%), it fails entirely on hardware (0%). Supplemental video suggests it exhibits volatile and jittery behavior."*
- [FACT — DPPO paper §5.3 verbatim]: *"To our knowledge, DPPO is the first RL algorithm to solve Transport from either state or pixel input to high (>50%) success rates."*
- [FACT — DPPO project page]: Furniture-Bench *Round-table* success rate improved from a low pre-trained baseline to 86 % from sparse reward only.

These results position trajectory-level diffusion-policy fine-tuning as the most promising route for drone obstacle avoidance, **conditional on the env being trustworthy**. Phase 1b builds that env.

This submission asks Zread to review the design decisions in `Phase1b_Gym_Wrapper_Design.md` **before W1 implementation begins**.

---

2. Items requiring Zread review

#	Item	Type	Rationale	Location
D1	**Action space = trajectory-level (8, 11)** (Option A) rather than low-level (11,) (Option B)	Core design	Preserves Phase 1a warm-start; aligns with DPPO theory	Design §5
D2	**EGO planner fully bypassed in Phase 1b** (no fallback)	Design / scope	Removes confound; safety fallback re-	Design §2, §7

#	Item	Type	Rationale	Location
			considered at Phase 2c	
D3	**PPO baseline before DPPO** (SB3 PPO with Phase 1a mean + fixed σ=0.05)	Methodology	Validate env separately from log-prob	Design §13 Q4
D4	**Reward composition: progress + arrival + time → +clearance/z → +smoothness** staged	Methodology	Avoid all-at-once reward-design failures	Design §8.2
D5	**Manual reward scale (no PopArt) in Phase 1b**	Methodology	Defer PopArt to Phase 2 multi-env mix	Design §8.3
D6	**Collision detection via `libgazebo_ros_bumper.so` with `gz sdf` post-processing**, polling-clr as cross-check	Implementation	Primary signal selection	Design §13 Q1
D7	**`deployment_action_steps = 4` in Phase 1b**, deferring 2-vs-4-vs-8 ablation	Hyperparameter	Match Phase 1a R2 deployment	Design §13 Q2
D8	**Lockstep at 1× initially, push to 2× after 100-episode stability burn-in**	Implementation	Avoid MAVLink timeouts	Design §13 Q7
D9	**Phase1aPolicyAdapter skeleton** — confirm `instantiate(cfg.policy)` + `load_state_dict` is the standard iDP3 ckpt-loading pattern	Implementation	API drift risk	Design §10
D10	**MLP vs UNet head question** — does Phase 1a R2's ckpt support fine-tuning with `act_steps < horizon_steps`?	Open question	Determines whether D7 is locked	Design §13 Q6
D11	**`/gazebo/reset_world` avoidance** — use `set_model_state` + selective entity reset to dodge the rotor-disappearance bug (rotors_simulator #311)	Implementation	Stability	Design §7
D12	**Reward weights initial values** in §8.1 — sanity check the magnitudes (esp. collision −100 vs progress ~+1)	Hyperparameter	Standard PPO scaling concern	Design §8.1
D13	**`max_steps = 80` env-steps cap** (≈ 3.2 min sim time at `deployment_action_steps = 4`)	Hyperparameter	May be too short for hard worlds	Design §9

3. 已知风险与不确定性 (from Design §13 + §14)

- **R-U1** [HIGH IMPACT]: MLP vs UNet head decision (D10). [FACT — DPPO README verbatim]: *"With fully-connected layers in MLP, empirically we find that using different chunk sizes for pre-training and fine-tuning with MLP leads to training instability."* Phase 1a R2 may need a head swap, or we accept `act_steps = horizon_steps = 8` for fine-tuning.
- **R-U2**: Gazebo bumper plugin reliability — documented URDF→SDF collision-name suffix bug (ROS Answers question 246448; iRobot Create 2 simulation blog).
- **R-U3**: `/gazebo/reset_world` instability after repeated calls (rotors_simulator #311; ROS Answers thread on actor plugin reset).
- **R-U4**: Whether `predict_action` exposes a clean log-prob hook for DPPO Phase 2 — not in scope for 1b but design-affecting.
- **R-U5**: PX4_SIM_SPEED_FACTOR cap. [FACT — PX4 docs verbatim]: *"Powerful desktop machines can usually run the simulation at around 6-10x, for notebooks the achieved rates can be around 3-4x."* We propose a conservative 2× cap.
- **R-U6**: Whether SB3 PPO's `MultiInputPolicy` handles our `Dict` obs (point cloud + state) without further customization. SB3 docs confirm Dict obs is supported via

`MultiInputPolicy` / `SimpleMultiObsEnv`, but our point cloud `(3, 4096, 6)` is a tensor shape stock `CombinedExtractor` may not encode well — likely need a custom feature extractor wrapping the iDP3 encoder.

---

4. Verdict requested per item

Same 4 categories as Phase 1a submission: **MUST_MODIFY** / **SUGGEST_MODIFY** / **OPTIONAL_MODIFY** / **PASS**, with per-item rationale.

Priority items (block W1 if MUST_MODIFY): **D1, D2, D6, D9, D10, D11**.

---

5. iDP3 / DPPO source-code cross-checks needed

CC#	Cross-check	Source
CC1	`predict_action` returns a dict with key `'action'` of shape `(B, n_action_steps, action_dim)` (not full horizon)	iDP3 `diffusion_policy/policy/diffusion_` (or whichever class Phase 1a instantiates)
CC2	`policy.normalizer['action'].unnormalize` API matches our adapter usage	iDP3 normalizer class
CC3	DPPO config field `horizon_steps` vs iDP3 config field `horizon` — alignment	irom-princeton/dppo `cfg/*.yaml`
CC4	DPPO recommendation — [FACT — DPPO README verbatim]: *"In our Furniture-Bench experiments we use `denoising_steps=100`, `model.ddim_steps=5`, and `ft_denoising_steps=5`"* — does Phase 1a R2's diffusion config support DDIM with `ddim_steps=5`?	iDP3 vs DPPO
CC5	[FACT — DPPO paper §4.3 verbatim]: *"Instead of fine-tuning the pre-trained model weights θ, we make a copy θ_ft — θ is frozen and used for the early denoising steps, while θ_ft is used for the last K' steps and updated with DPPO."* — confirm iDP3 policy class allows this `θ` / `θ_ft` split	DPPO paper §4.3
CC6	[FACT — DPPO paper §4.3 verbatim]: *"clipping σ_k to a higher minimum value (denoted σ^exp_min, e.g., 0.01–0.1) when sampling actions helps exploration."* — pre-record for Phase 2 hyperparams	DPPO paper §4.3
CC7	iDP3 `SequenceSampler` honors `pad_before` / `pad_after` deterministically given a seed — for Phase 1b sample-weight ablation reproducibility	iDP3 sampler

---

6. Decision dependencies

This submission's verdicts cascade into the W1–W4 implementation plan:

If verdict	Then implementation action
D1 MUST_MODIFY (force Option B)	Re-pretrain Phase 1a with `action_dim=11`, single-step output — adds ~2 weeks; blocks Phase 1b
D2 SUGGEST_MODIFY (keep EGO as fallback)	Add safety-shielding layer; impacts step semantics §6
D6 MUST_MODIFY (rule out bumper)	Switch to IMU-spike + polling-clr hybrid as primary; impacts §9 termination logic
D10 returns "MLP must keep T_a = T_p = 8"	Update `deployment_action_steps = 8` in §13 Q2 and §6; impacts reward smoothness term
Other items MUST_MODIFY	Update in place; no schedule slip

==========

交付物结束（共 4 个文件）